

GoComet Nova: Product Requirements Document (PRD)

1. Executive Summary & Vision

What is GoComet?

GoComet is an industry-leading supply chain visibility and automation platform utilized by enterprises across 30+ countries. It manages the complexities of global logistics—tracking shipments across ocean, air, road, and rail, while handling procurement, contracts, invoices, and compliance. However, moving \$12 trillion in global goods annually requires more than just tracking; it requires autonomous action.

What is Nova?

Nova is GoComet's platform transformation. It is not just another dashboard or chatbot; it is an AI-native operating layer and orchestration engine. Nova powers specialized AI agents (like Incident Lens for SLA monitoring, Viera for conversational analytics, and GoVista for inbox automation) that execute governed, multi-step workflows. While traditional SaaS platforms act as a **System of Record** (a passive database where humans log what happened) or a **System of Engagement** (a UI where humans talk to each other), Nova acts as a **System of Outcomes**. It doesn't just show a user that a document has an error; it autonomously extracts the data, cross-validates it against customer-specific rules, flags the exact discrepancy, and drafts the amendment email to the supplier.

The Forward Deployed Engineer (FDE) Model

To deliver Nova effectively, GoComet utilizes the **Forward Deployed Engineer (FDE)** model (pioneered by companies like Palantir). Global logistics is messy—every customer has unique rules, legacy systems, and exception-filled processes. Standard "out-of-the-box" SaaS fails here. The FDE model bridges the "last mile" gap of AI. As an FDE, engineers don't just build abstract microservices; they sit directly with enterprise clients, understand their chaotic processes, and write production code (workflows, rules, and agent prompts) to solve their specific problems. This ensures Nova adapts to the client's reality, rather than forcing the client to adapt to rigid software.

2. Problem Statement & Real-World Context

Global trade is heavily reliant on documentation (Bills of Lading, Commercial Invoices, Packing Lists, Certificates of Origin). Research shows that **documentation errors contribute to 60–70% of shipping delays**.

The Real-World Cost of Errors

When documents are processed manually via email attachments, human fatigue leads to clerical errors. These mistakes trigger a costly "domino effect":

1. **Demurrage & Detention (D&D) Charges:** If customs clearance is delayed due to incorrect paperwork, containers sit at terminals beyond their free time. D&D fees range from **\$75 to \$300 + per container per day**, adding **10–25% to total shipping costs**.
2. **Customs Holds & Penalties:** An incorrect HS Code or missing Certificate of Origin will flag a shipment for regulatory inspection, causing days of delay and potential legal penalties.
3. **Administrative Overhead:** Fixing an error requires an "amendment cycle" (emailing the supplier, waiting for a fix, re-verifying). A single amendment cycle can delay a shipment by 24–72 hours, consuming the Cargo Group's bandwidth.

Specific Failure Modes (What breaks today)

1. **The Omission:** A supplier forgets to attach the Packing List. The Cargo Group (CG) operator misses this in a crowded inbox. The cargo arrives, but customs refuses clearance.
2. **The Inconsistency:** The Gross Weight on the Bill of Lading (4,250 KG) does not match the Commercial Invoice (4.25 MT). A human might silently approve this, but a strict customs officer will flag it.
3. **The Rule Violation:** A customer's specific SOP dictates that the Incoterm must be "DDP", but the supplier submits "FOB".

Success in the first 5 minutes: A CG operator logs in, sees a dashboard of shipments automatically categorized by status. They click on a "Flagged" shipment, instantly see the highlighted discrepancy (e.g., "HS Code mismatch"), review the AI's pre-drafted amendment email, and click "Send". What used to take 15 minutes of manual checking now takes 10 seconds.

3. Personas & Jobs-to-be-Done (JTBD)

Persona 1: Sarah, the Cargo Group (CG) Operator

Sarah is the validator. She receives emails from suppliers, downloads PDFs, and checks every field against customer rules stored in her head or in spreadsheets. She is overwhelmed by volume and amendment cycles.

- **JTBD 1:** When a new shipment email arrives, I want the system to instantly extract and validate all attached documents, so that I don't have to manually open and read PDFs.
- **JTBD 2:** When a document contains a discrepancy, I want to see exactly what was found versus what was expected, so that I can quickly verify the error without hunting through the PDF.
- **JTBD 3:** When I need to request a correction, I want a pre-drafted email detailing the exact errors, so that I can click "Send" instead of typing out repetitive amendment requests.

Persona 2: Michael, the Supplier / Shipper (SU)

Michael dispatches goods and generates the documents. His job feels done when he sends the email, and he hates getting amendment requests 3 days later.

- **JTBD 4:** When I submit my shipping documents, I want immediate, clear feedback on any errors, so that I can fix them before the cargo physically departs and it becomes expensive to amend.

Persona 3: David, the Forward Deployed Engineer (FDE)

David sets up Nova for new enterprise clients.

- **JTBD 5:** When configuring a new customer, I want to define deterministic validation rules (e.g., 2% weight tolerance), so that the AI rigorously enforces business logic without hallucinating.
-

4. Agent Architecture: The Technical Core

We employ a **Multi-Agent Architecture** pipeline: `Extractor Agent → Validator Agent → Router Agent`.

Why Three Agents? Why not one giant prompt?

A single giant prompt ("Here is a PDF and a list of rules, tell me if it's approved and write the email") is catastrophic in enterprise environments. It conflates *reading* (probabilistic) with *math/logic* (deterministic), leading to hallucinations, silent approvals, and impossible debugging. By enforcing sharp agent boundaries, we align with the planner/executor/verifier pattern:

1. **Extractor Agent (Executor):** Sole responsibility is to read the PDF and output structured JSON with confidence scores. It uses a single vision model pass to maintain spatial layout context, which is critical for merged cells and complex tables.
2. **Validator Agent (Verifier):** Sole responsibility is to compare the JSON against customer rules. It operates using a hybrid approach: Layer 1 is deterministic (zero LLM cost) for exact matches and tolerances; Layer 2 uses fuzzy matching/embeddings; Layer 3 falls back to the LLM for complex semantic matching and unit conversions.
3. **Router/Decision Agent (Planner):** Sole responsibility is to read the validation output and decide the outcome (Approve, Flag, Draft Email). The decision logic is 100% deterministic to ensure safety (e.g., Any CRITICAL mismatch -> amend), while the email drafting uses the LLM for professional business writing.

State and Orchestration

Agents communicate via **structured handoffs** orchestrated by **LangGraph**.

- **Crash Survival & State Management:** LangGraph maintains a state graph using a SQLite-backed checkpointer (migratable to Postgres for production). If the server crashes during extraction, the state is persisted. Upon reboot, the pipeline resumes exactly at the failed node.
 - **Human-in-the-Loop:** LangGraph's `interrupt_before` functionality inherently supports pausing the pipeline after validation, presenting results to the Cargo Group (CG) operator, and waiting for their review before sending any emails.
-

5. LLM & Tooling Choices

- **LLM (Extraction, Validation & Routing):** **claude-sonnet-4-6**.
 - *Why:* claude-sonnet-4-6 offers best-in-class vision capabilities for extraction

(including native PDF support), and extremely reliable structured output and tool-calling capabilities. It strikes the ideal balance of accuracy (> 95% on clean documents) and latency.

- *Fallback*: If the document scan is entirely illegible, the model returns a low confidence score, which the Validator catches and routes directly to Human Review.

- **Orchestration: LangGraph.**

- *Why*: We need cyclic graphs (for retry loops if extraction fails schema validation) and durable state persistence. It also easily supports extensibility without rewriting nodes.

- **Storage & Query Layer (Text-to-SQL)**: SQLite for the database, combined with a natural language query layer using GPT-4o-mini to convert user questions into SQL via schema-aware prompting.
 - **Structured Output & Tool Calling**: We aggressively use tool calling in the Extractor (for JSON schema enforcement), the Validator (for delegating deterministic math/fuzzy logic to python tools), and the Router to ensure perfectly typed data throughout the pipeline.
-

6. Trust, Failure Handling & Evals

Stopping Hallucinations

To prevent the agent from inventing missing values:

1. **Strict Prompting + Fallbacks**: The prompt strictly dictates returning `null` if a field is missing.
2. **OCR Verification Layer (DocTR/PyMuPDF)**: We use a lightweight OCR pass as a verification layer. If the LLM extracts an Invoice Number that does not exist anywhere in the raw OCR text, it is flagged as a potential hallucination and the confidence score is forcibly dropped to 0.3.

Handling Low Confidence & Severity-Based Routing

- **No Silent Approvals**: The Extractor must emit a confidence score (0.0-1.0) for every field. If `confidence < 0.85`, the Validator marks the field as uncertain. Any uncertain fields bypass auto-approval and force human review.
- **Severity Classification**: Validation mismatches are classified by severity (e.g., CRITICAL for wrong consignee, HIGH for weight mismatch > 5%, MEDIUM/LOW for minor formatting). Any CRITICAL mismatch strictly mandates an amendment request. Any HIGH mismatch forces a human review.
- **Runaway Costs**: LangGraph is configured with a `recursion_limit` of 5. If the Extractor fails to produce valid JSON after retries, the pipeline aborts, marks the status as `SYSTEM_ERROR`, and halts to prevent infinite LLM billing loops.

Evals

- **Offline Eval**: A test suite runs over a Golden Dataset of 100 historical shipments (clean, messy, missing docs). We measure **Extraction F1 Score** (Precision/Recall on fields) and **Validation Accuracy** (Did it catch the deliberate errors?).

- **Online Metric: Human Edit Distance.** For every drafted amendment email, we track the Levenshtein distance between the AI's draft and the final text the human operator actually sends. If the distance grows, our router prompt needs tuning.
-

7. Metrics & Success Criteria

North-Star Metric

Straight-Through Processing (STP) Rate: The percentage of shipments processed, validated, and approved end-to-end without any human intervention.

Supporting Metrics

1. **Time-to-Action:** Average time from receiving the supplier's email to the CG operator clicking "Approve" or "Send Amendment" (Target: < 2 mins).
2. **Human Edit Distance:** Percentage of AI-drafted emails sent without manual edits (System Quality).
3. **Extraction Confidence Average:** Tracks document quality and OCR health over time.
4. **False Negative Rate:** Percentage of shipments auto-approved that contained an error (Must be strictly 0%).
5. **Cost per Shipment:** Total token cost per pipeline execution (Target: < \$0.05).

Go / No-Go Criteria for a 2-Week Customer Pilot

To graduate from staging to a live 2-week pilot with a customer, the system must prove:

1. **Zero False Negatives:** It must *never* silently approve a non-compliant document across a 500-document historical backtest.
 2. **STP > 20%:** At least 1 out of 5 shipments must require zero human touches.
 3. **UI Latency:** The CG Validation screen must load and render discrepancy states in under 2 seconds.
-

8. What's Next? (After Part 1 Ships)

If we had two more weeks, we would build:

1. **Inbox Trigger Integration (Part 2):** Connect the pipeline to a live email inbox (e.g., via Microsoft Graph API or Gmail Pub/Sub) to auto-trigger the LangGraph pipeline the second a supplier emails documents.
2. **Cross-Document Consistency Checks:** Implement logic in the Validator to ensure that fields shared across documents (e.g., Gross Weight on the BOL vs. the Packing List) match perfectly, highlighting discrepancies between attachments.
3. **Interactive Feedback Loop:** Add a feature in the UI where if a CG operator corrects an extracted value, that correction is logged to a vector database (Weaviate) to few-shot prompt the Extractor agent on future runs for that specific supplier format.